

➤ TDA Introducción

- **TDA**: Es un modelo que define un conjunto de datos y las operaciones que se pueden realizar sobre ellos, sin especificar la implementación. Además, un TDA separa la interfaz de la implementación.

• Características:

- **Abstracción**: Los usuarios del TDA interactúan con él a través de su interfaz sin preocuparse de los detalles internos.

- **Interfaz**: Define las operaciones que se pueden realizar sobre el TDA, como insertar, eliminar, buscar, etc...

- **Implementación**: Existen diversas formas de implementarlo sin afectar su funcionamiento.

- Tipos de TDA:

• **Lista**: Colección ordenada de elementos, con operaciones como agregar, eliminar, o buscar.

• **Pila (stack)**: Colección con acceso al último elemento (LIFO) operaciones push (agregar) y pop (eliminar).

• **Cola (Queue)**: Colección con acceso por el principio (FIFO) operaciones enqueue (agregar) y dequeue (eliminar).

• **Conjunto**: Colección de elementos únicos con operaciones de unión, intersección, etc...

• **Diccionario (Map)**: Colección de pares clave-valor, con operaciones clave de búsqueda, intersección y eliminación.

- TDA de lista simplemente ligada.

Estructura de datos donde los elementos, llamados nodos, se organizan en una secuencia. Cada nodo contiene un valor y una referencia (enlace) al siguiente nodo en la secuencia, formando así una cadena o lista.

- **Estructura secuencial:** Los elementos se almacenan en un orden específico.
- **Nodos:** Cada nodo, a excepción del último, tiene un único enlace que apunta al siguiente nodo de la lista.
- **Final de la lista:** El último nodo tiene un enlace que apunta a null o un valor especial que indica el final de la lista.
- **Acceso secuencial:** Para acceder a un elemento específico, es necesario recorrer la lista desde el principio para encontrarlo, lo que puede ser menos eficiente que el acceso directo en otras estructuras como los arreglos.

Ejemplo.

```
#include <iostream>
using namespace std;
```

```
class Nodo {
public:
    int dato;
    Nodo* siguiente;
    Nodo(int valor) {
        dato = valor;
        siguiente = nullptr;
    }
};
```

```
int main() {
    Nodo* primero = new Nodo(10);
    Nodo* segundo = new Nodo(20);
    Nodo* tercero = new Nodo(30);

    primero->siguiente = segundo;
    segundo->siguiente = tercero;
    Nodo* actual = primero;
    while (actual != nullptr) {
        cout << actual->dato << "-> ";
        actual = actual->siguiente;
    }
    cout << "NULL" << endl;
    return 0;
}
```